Thesis Defense

Facetwork: A Language-Directed, Lock-Free Model for Live-Updatable Distributed Workflow Execution

Candidate's opening statement

I. Foundations and correctness

- Q1 (Prof. Abadi)
- Q2 (Prof. Abadi)
- Q3 (Prof. Kleppmann)
- Q4 (Prof. Gray, represented by his published work)
- Q5 (Prof. Kleppmann)

II. Language design

- Q6 (Prof. Pike)
- Q7 (Prof. Liskov)
- Q8 (Prof. Pike)
- Q8a (Prof. Pike)
- Q8b (Dr. Yegge)
- Q9 (Prof. Pike)

III. The coordination protocol

- Q10 (Prof. Gray)
- Q11 (Dr. Yegge)
- Q12 (Prof. Kleppmann)
- Q13 (Prof. Gray)
- Q14 (Prof. Abadi)

IV. Recovery and determinism

- Q15 (Prof. Liskov)
- Q16 (Prof. Kleppmann)
- Q17 (Prof. Gray)

V. Comparisons

- Q18 (Prof. Liskov)
- Q19 (Dr. Yegge)
- Q20 (Prof. Kleppmann)
- Q21 (Prof. Pike)

VI. Evaluation

- Q22 (Prof. Kleppmann)
- Q23 (Dr. Yegge)
- Q24 (Prof. Abadi)

VII. Scope, future work, and impact

Q25 (Dr. Yegge)
Q26 (Prof. Liskov)
Q27 (Dr. Yegge)
Q28 (Prof. Pike)

Candidate's closing statement

Addendum (October 2026): answers that no longer stand, and the questions I would now expect

Corrections

Questions I would now expect

Q29 (Prof. Kleppmann)
Q30 (Prof. Gray)
Q31 (Prof. Abadi)
Q32 (Prof. Liskov)
Q33 (Dr. Yegge)

Thesis Defense

Facetwork: A Language-Directed, Lock-Free Model for Live-Updatable Distributed Workflow Execution

Candidate: Claude (Opus 4.6, 1M context) **Supervisor:** Ralph Lemke **Date of defense:** 2026-04-13

Committee:

- **Prof. M. Abadi** (chair) — distributed computing theory, foundations.
- **Prof. R. Pike** — programming languages, systems.
- **Prof. J. Gray** (in memoriam, represented by his written work) — databases and transactions.
- **Prof. M. Kleppmann** — streaming systems, data intensive applications.
- **Dr. S. Yegge** — industrial practitioner, external examiner.
- **Prof. B. Liskov** — abstraction, language design.

The fictional composition reflects the range of objections I expect the work to face. No endorsement from any named scholar is implied or should be inferred.

Candidate's opening statement

I will not restate the thesis. The committee has read it. I want to say three things before questions.

First, the thesis has an argumentative structure, not a descriptive one. I am not claiming Facetwork is uniformly better than every alternative. I am claiming that its four design choices — a typed DSL, document-atomic coordination, state-persisted recovery, and live operational control — reinforce each other in a way that serves a specific class of workload, which I call *live long-running domain workflows*, better than any single alternative in the literature.

Second, much of the work is synthesis. The lock-free claim protocol is a straightforward application of atomic document updates; the DSL borrows from CWL, Nextflow, and BPEL; the lease model is Gray and Cheriton. The contribution lies in the combination and in two small-but-genuine new mechanisms: the staged timeout budget and the block-mediated evaluator.

Third, the honest limitations are in Chapter 15. I am not going to hide behind them in the questions that follow, and I am not going to pretend they do not exist.

I look forward to the questions.

I. Foundations and correctness

Q1 (Prof. Abadi)

> *Your safety argument in §5.5 leans on MongoDB's single-document linearisability. That is a recent guarantee; for most of MongoDB's existence the ceiling was "majority write concern" with weaker semantics. First, how exactly do you state your safety property? Second, what does the FLP impossibility result tell us we cannot achieve — and are you quietly assuming something FLP forbids?*

The safety property, stated precisely, is: for each workflow step, at most one task document exists in state `running` at any real-time instant, and at most one handler is believed by the system to hold that task's lease. I am not claiming that at most one handler is *executing* at any instant — a crashed runner's process may still be running C-extension code until its OS reaps it. I am claiming that at most one handler's output is *accepted* by the system.

The property relies on three guarantees from MongoDB. First, that `find_one_and_update` is atomic at the document level under the default write concern. Second, that a partial unique index rejects conflicting concurrent upserts. Third, that under `majority` read and write concerns, a committed write is observable by all subsequent reads on a primary and by all subsequent reads on secondaries after sync. These have been formally specified since MongoDB 4.0's multi-document transaction work, and the single-document path has always been atomic; what was weaker historically was *multi-document* consistency, which Facetwork does not use.

FLP rules out a *deterministic* consensus protocol that always terminates in asynchronous systems with even one faulty process. Facetwork does not run consensus in the strict sense. It offloads agreement to MongoDB's replica-set protocol, which itself runs consensus under partial-synchrony assumptions (the same assumptions Raft makes), and it uses lease-based timeouts for liveness. This is precisely the standard move to sidestep FLP: assume partial synchrony, use timeouts as the liveness proxy. I am not quietly assuming FLP-forbidden things; I am explicitly in the partial-synchrony camp and the thesis should probably have said so more prominently.

Q2 (Prof. Abadi)

> *You repeatedly say "lock-free." In the formal literature "lock-free" means a non-blocking progress condition: in any execution, at least one process makes progress in a bounded number of its own steps. Facetwork uses atomic operations on MongoDB and lease timeouts. Does it satisfy lock-freedom in the formal sense, or are you using the term colloquially?*

Colloquially, and I should have flagged this. The formal property I want is closer to **non-blocking at the application layer**: no application-level mutex, no coordinator process whose failure stalls progress, no leader election that requires a quorum of application nodes. MongoDB's internal primitives — the replica-set consensus, the WiredTiger engine's locks — are obviously blocking at their own layer.

Under the stronger Herlihy/Shavit definition, Facetwork is lock-free in the degenerate sense that each application-layer atomic operation (the `find_one_and_update`) is a single database call that either succeeds or fails bounded in time by the database's internal progress properties. The composed claim protocol inherits this bound transitively. It would be correct to say Facetwork is *lock-free at the application layer* and *wait-freedom-preserving for the underlying store*.

I accept that the thesis should distinguish this more carefully, and the revised final version will.

Q3 (Prof. Kleppmann)

> *Your lease-based reclamation is identical to Gray and Cheriton 1989. The partial unique index is just a constraint. What is genuinely new?*

Two things, and I want to be precise about neither being a revolution.

First, the combination of lease-based reclamation with a partial unique index on `(step_id, state=running)` is, to my knowledge, not published. It is the partial unique index that makes the claim protocol robust against any implementation subtlety: even if the atomic update had some edge case I had not anticipated, the unique index would reject the second `running` document. This is defence-in-depth, cheap to add, and it has the property that correctness is visible from the schema, not only from the protocol code. I have not seen this pattern explicitly documented before, though I suspect it has been discovered independently in production systems.

Second, and more genuinely novel: the **staged timeout budget** (§8). Every workflow system has per-task timeouts. None, to my knowledge, expose a nested, dynamically extensible, handler-declared stage budget that composes with a global safety ceiling. I believe this is a real contribution, even though it is mechanically simple: it takes a sequence of existing primitives (lease renewal, heartbeats, per-task timeouts) and gives the handler author a typed context manager that does the right thing at stage boundaries. The contribution is in the API shape and in its composition with the global ceiling — not in any single atomic primitive.

If the committee wants me to withdraw any novelty claim for the claim protocol itself, I will — and I will sharpen the novelty claim for the staged timeout model.

Q4 (Prof. Gray, represented by his published work)

> *You have a zombie-handler scenario you call "a bug we accept." Explain that more carefully. What are the failure modes, and what do you propose for handlers that have billable external side effects?*

The scenario is this. Handler H1 claims a task and begins executing. Partway through, the process or network connecting H1 to MongoDB fails such that H1 cannot renew its lease. Another runner, H2, observes the lease as expired and reclaims the task. H1 is not dead; it is only unreachable. H1 eventually succeeds at whatever external work it was doing — perhaps sending a payment — and attempts to write its result back to MongoDB. H2 is simultaneously doing the same work, because the task appears to it to be unheld.

In the current design, H1's result write uses `update_task_state`, which is not gated on `server_id` or on `lease_expires`. It will therefore overwrite H2's result (or be overwritten by H2's, depending on ordering). The task is marked complete either way, and from the workflow's perspective everything is fine. The external side effect, however, has happened twice.

I accept that this is unacceptable for payments. For such handlers the industry-standard solution is **external idempotency keys**: each task is given a deterministic key derived from its UUID, and the external API (Stripe, Square, etc.) accepts the key and refuses to execute the side effect twice. Facetwork should provide, and does not yet, a framework-level idempotency-key utility and an `@idempotent` handler annotation that causes the runner to threading the key through automatically. I flag this in §14.2.

What Facetwork *does* provide is a principled alternative: gate the result write on the current `server_id` and `lease_expires`, so that H1's write fails after its lease has expired and is never committed. This is a three-line change I will add before the thesis is submitted in final form. It does not solve the external-side-effect duplication problem — that requires idempotency keys — but it solves the *internal* state inconsistency problem, which is the committee's real objection. I appreciate the push.

Q5 (Prof. Kleppmann)

> *Heartbeat cost. A runner with 100 concurrent tasks, heartbeating each every 10 seconds, generates 10 writes per second. A fleet of 100 such runners is 1,000 writes per second to the task collection. At what fleet size does the heartbeat load saturate a single-primary MongoDB deployment?*

For a single-primary WiredTiger MongoDB on modern commodity hardware — say an AWS m6i.2xlarge, which is generous for a coordinator-only database — the write throughput to a single collection is in the low thousands per second before latency degrades noticeably. Your 1,000-write example sits at about 25–35% of that ceiling. At 3–5x that load, write amplification in the journal and the WAL competes with application I/O and latency spikes begin.

So practically: the system as described scales cleanly to a few hundred runners with 100 concurrent tasks each. Beyond that, one mitigates in two ways. First, reduce heartbeat frequency for tasks with long expected durations: a 30-minute stage does not need a 10-second heartbeat. Second, shard the task collection by a high-cardinality key (such as `runner_id`) so that heartbeats distribute across shards. MongoDB sharding is mature, and the claim protocol preserves its semantics under sharding as long as the unique index is sharded with the appropriate hash or range key.

At the scales I describe in §14.3 — fleets of up to a dozen runners across a few hosts — the heartbeat load is a small fraction of the ambient workload. At fleet sizes 100x larger, operational tuning is required. The design is not broken at that scale; it is untested.

II. Language design

Q6 (Prof. Pike)

> *FFL has no user-defined generics, no higher-order facets, no general recursion. Nextflow and CWL are both more expressive. On what grounds do you claim FFL is a language worth learning?*

On three grounds.

First, every feature I have omitted from FFL is a feature that, in the languages that have it, interacts non-trivially with the runtime's static checking. Generics require type-variable resolution; higher-order facets require first-class function values in the graph; recursion requires unbounded graph expansion. Each of these moves more responsibility from the compiler to the runtime, which is exactly the direction I argue against in §4.10. FFL is small on purpose. A language that can be fully type-checked at compile time offers a specific, concrete benefit — the dashboard can render the graph, the compiler can detect unreachable branches, the IDE can complete identifiers — that a larger language forfeits.

Second, the workloads I have in mind do not need the features Nextflow provides. Genomic pipelines in practice are chains of typed steps with data-flow dependencies; I have looked at several, and not one requires user-defined generics or recursion. Nextflow's expressiveness is a cost its users pay in tooling (the grammar is complex) and in runtime behaviour (workflows can be dynamically generated at runtime) that a simpler language would spare them.

Third, expressiveness is not a virtue in isolation. The question is *who writes workflows in this language*. For domain experts — epidemiologists, urban planners, SREs — the minimum viable set of constructs is small: typed steps, sequencing, conditional, iteration over a bounded collection. FFL has exactly those. A language with more is a language that has more corners for its users to get lost in.

I will concede that CWL's file-object type system is more developed than FFL's — it carries `File` as a first-class type with secondary files and formats, and FFL passes paths as strings.

I previously conceded Snakemake's wildcards as well, and I now think that concession was wrong, so let me withdraw it and say why. Wildcards do two separable things. The first is **enumeration**: `glob_wildcards("data/{sample}.txt")` discovers the sample set, and `expand` turns it into targets. That has a direct analogue in FFL and it is not hypothetical — it is how the largest fan-out in this project is written:

```
children = county.atlas.ListCounties(prefix = $.prefix, bucket = $.bucket)
    andThen foreach cty in $.counties limit $$.concurrency { … }
```

`ListCounties(prefix, bucket)` *is* a glob; noaa-weather's `DiscoverStations → foreach station` is the same shape. Enumerate, then iterate. Nothing is lost.

The second thing is **backward unification**: given `input: "data/{sample}.txt"` and `output: "out/{sample}.csv"`, asking for `out/A.csv` lets Snakemake infer `sample=A` and derive the chain that produces it. That genuinely has no FFL analogue. But it is not really an ergonomic feature of path syntax — it is make-style backward chaining, and it is inseparable from the model that outputs are files, that filenames encode parameters, and that existence plus mtime implies completeness. §13.3 gives a measured demonstration of that last assumption failing silently. Conceding backward chaining as a loss concedes something welded to a premise the thesis argues against.

There is a sharper point than ergonomics, which is that the wildcard mechanism is bound to a local POSIX filesystem, and the workloads this thesis targets do not keep their data there. Fleet data lives in MinIO, in HDFS, or as rows in Postgres. A key in an object store is not a path — object stores have prefixes and a listing API, not directories — and a database record has no filename to match at all.

What happens when you point the mechanism at that data is worth stating precisely, because I tested it rather than assuming. `glob_wildcards("s3://bucket/data/{sample}.txt")` returns an empty list. Not an error, not a warning — empty. Feed that to `expand()` and the workflow builds zero jobs, and Snakemake 9.25 reports:

```
1 of 1 steps (100%) done          exit code 0
```

Complete success, zero samples processed. That is not a wrong answer; it is *no work at all*, reported as done — the same silent-success failure family as the stale-output result in §13.3, and arguably the more dangerous member of it.

This is not a claim that Snakemake cannot use object storage: version 8 introduced storage plugins, and remote inputs are declared through them. It is a narrower claim, and it is the one that matters for the concession I withdrew above. **The discovery half of wildcards — the half that has ergonomic value — is a local-filesystem operation.** Move the data to a bucket and you write the enumeration yourself, in Python, against a listing API. The ergonomic advantage evaporates precisely in the setting this thesis is about — while on the FFL side the equivalent (`fw.file.List`, below) is one built-in call whose backend follows the URI scheme.

I should not overstate the contrast. An FFL enumeration step that returns an empty collection also fans out over nothing, and the workflow also completes. Empty is not impossible here either. The differences are where the enumeration happens and whether its result is observable: `ListCounties` runs on a runner that holds the store's credentials and can actually see the data, so an empty result means the prefix is empty rather than that the wrong machine was asked; and the result is a value recorded in the step store, so `children.counties = []` is visible in the dashboard rather than being an absence inferred from a DAG that was never built. A handler can also refuse it outright — the fan-in in `experiments/state-capitals` raises rather than emit a table of 47 states, because a partial result that looks like a result is worse than a failure.

There is a genuine trade in the *other* direction that I should state, because it favours FFL for the workloads this thesis targets. `glob_wildcards` is evaluated during DAG construction, on the submitting machine, before any rule runs — so it sees what that machine's filesystem sees. `ListCounties` is a **step**: it executes on a runner that can reach the object store, its result is a durable typed value that flows to the `foreach`, it is retryable, and the enumeration it performed is recorded in the step. For a fleet whose data lives in MinIO rather than on the laptop that launched the run, enumeration has to be work rather than parse-time metaprogramming.

I claimed, in an earlier version of this answer, that a cost remained: FFL's version needs someone to write `ListCounties` — a facet and a handler — where Snakemake's needs one line, which is real friction for a small pipeline. **That is no longer true, and the fix is the interesting part.** Enumeration is not domain logic; it is infrastructure, and it now ships with the framework:

```
found = fw.file.List(path = $.dir, pattern = "*.csv")
    andThen foreach p in found.paths { … }
```

`fw.file.*` is a set of built-in facets — `List`, `Stat`, `Hash`, `ReadText`, `WriteJson`, `Copy` and the rest — registered by every runner at startup, with no handler for anyone to write. One line, as Snakemake's is.

The difference that survives is the one this section is about. `fw.file.List` resolves its path through the storage abstraction, so `path` may equally be a directory, an `s3://` prefix or `hdfs://`, and the workflow does not change. `glob_wildcards` pointed at `s3://…` returns an empty list and the run reports success having done nothing. So the comparison is no longer "one line versus a facet and a handler"; it is one line that works where the data is, against one line that works on the machine that launched the run.

There is a genuine cost left, and it is smaller and different: a built-in library is a surface the framework now owns and must keep coherent. The registration cross-checks the shipped FFL against the handler dispatch and refuses to start on drift, because the failure it replaces — a facet that compiles, dispatches, and dies on a runner with "no handler" — is exactly the kind of late, distant error this thesis argues against elsewhere.

Since that answer was written the library has grown to twenty-one facets in four namespaces, and I should report what growing it taught me, because two of the three lessons are unflattering.

The selection rule held, and it is narrow: a facet earns its place when a survey finds the same non-domain logic rewritten across many domains *and getting it wrong differently each time*. `fw.http` qualified — fourteen of twenty-three domains hand-rolled download-plus-cache, one frozen forever on a moving `…-latest` target, one recording a cache key it never compared. `fw.archive` qualified — nine domains, seventeen sites. Both are cases where §13.3's argument had been quietly lost inside my own system, which is the strongest evidence I have that "leave it to the handler author" is a real cost rather than a rhetorical one.

The first unflattering lesson is that adding ambient declarations exposed two latent defects that no test covered: `use fw.file` was rejected as an unknown namespace, and the runtime resolved facets only against a workflow's stored AST — which, by construction, does not contain facets that ship with the framework. The first had been failing a test since the day `fw.file` landed and I had not noticed; the second only appeared when a facet was exercised on a real runner rather than in unit tests. Both were consequences of the design decision, not accidents of implementation, and I did not anticipate either.

The second is about `fw.exec`, which invokes an external command and is the direct analogue of Snakemake's `shell:`. I built it and I want to be candid that it fails the selection rule above: all forty-eight `subprocess` sites in the domain packages sit inside Python handlers, where `subprocess` is already available, so it removes no duplication whatever. Its justification is reach for an author who writes no Python — which is a real justification, but a weaker one than the other three namespaces have, and it deserves to be labelled as such rather than presented as more of the same.

It also forced an asymmetry with `shell:` that I would rather not need. `Run` takes an argv list and never invokes a shell, which is a plain improvement — Snakemake interpolates wildcards into a shell string, and a filename containing a space or `$(…)` changes what runs. But `Run` also requires the command to be named in the runner's `FW_EXEC_ALLOW`, and unset means refuse. Snakemake needs no such gate because the person writing the Snakefile owns the machine. Facetwork had already assumed otherwise — the `script { }` sandbox removes the `open` builtin and allowlists imports precisely because the FFL author is not fully trusted — and an ungated exec would have erased that in a single facet. The cost is that a workflow which would run anywhere under Snakemake needs a provisioned host here, and I do not think "it works on the host that was set up for it" is as good a property as "it works". I take the trade because the alternative is a security boundary that exists in one part of the language and not in another, which is worse than either consistent choice.

Q7 (Prof. Liskov)

> *Your thesis argues for separation of concerns between workflow topology and handler implementation. But Python libraries with decorators achieve the same separation — the Temporal SDK lets me write a workflow that calls activities by name. Why the whole DSL?*

The separation in Temporal's SDK is a programmer discipline, not a language guarantee. Nothing in Python prevents a workflow function from importing an activity's implementation module directly, holding a reference to a mutable object from that module, or otherwise coupling to the implementation in ways the SDK's type system cannot detect. In practice, Temporal workflows regularly accumulate these leaks, and the community has a specific term — "workflow-activity bleed" — for the refactor that fixes them.

In FFL, the coupling *cannot* happen. A workflow references facets by qualified name. There is no import of handler modules from workflow code — the two are literally in different source files of different languages. The handler's implementation can be replaced at registration time without the workflow changing, and the compiler verifies that this is true because the workflow's only knowledge of the handler is its declared signature.

The question is whether this guarantee is worth the cost of a new language. I argue yes, because the guarantee enables downstream features that are otherwise impossible. Live handler updates at runtime work because the workflow has no compile-time coupling to the handler implementation. Multi-language handler registration works for the same reason — Go, Scala, and Java handlers cannot be imported by a Python workflow, and the workflow should not be trying. The DSL enforces the property that everything else in the system depends on.

It is a design with a clear principle: guarantee the property that enables the other features, even if the guarantee costs a language. If the committee believes a Python library could offer equivalent guarantees, I welcome the discussion.

Q8 (Prof. Pike)

> *The `$.attribute` syntax looks like Perl. It is visually noisy and semantically peculiar. Defend it.*

It is visually noisy and I dislike it as much as you do. I will say what it buys.

Without `$.`, the alternative is to give outer-container attributes the same namespace as block-local bindings. This creates shadowing: an inner `region = ...` would shadow the outer `region: String` parameter. Shadowing is the source of an enormous number of bugs in imperative languages and I did not want to pay for that in FFL. The alternatives are (a) prohibit shadowing, which is restrictive and unnatural, or (b) use explicit syntactic distinction between "my scope" and "my container's scope," which is what `$.` does.

Better-known languages make similar choices. Scala's `this.` disambiguates class attributes from locals. JavaScript's explicit `this.` does the same. Python's `self.` is the same pattern. `$.` is Facetwork's version, and I picked `$` because it is visually distinct from identifier characters and cannot appear as a bare word in any reasonable surface syntax. Since this defense the sigil has stayed but its meaning has sharpened: what was a single flat container level is now a *relative* scope, in which `$` names the immediate container and `$$`, `$$$` walk outward. I had expected any change here to be cosmetic — swap `$.` for `outer.`, semantics untouched. The opposite happened: the syntax survived and the semantics moved. I take that redesign up in Q8a.

I accept that `$` carries Perl baggage for readers of a certain age. The syntax is stable now and, on the evidence of the examples I have shown, authors acclimate to it quickly.

Q8a (Prof. Pike)

> *You have just told us the scoping model changed after the defense. That is unusual — a language's scope rules are its spine. Walk us through what changed, and then justify the up-level operators `$$` and `$$$`. They look like exactly the Perl-noise you apologised for a moment ago.*

The old model was flat. Every block resolved names against one implicit container — the workflow's inputs — plus its own local bindings, and `$.` reached that single container level. It worked for shallow workflows and it was easy to explain. It was also a lie about the structure of the programs authors were actually writing, because facets and steps nest, and a flat model hands every nesting level the same one outer scope.

The relative model tells the truth about nesting. `$` names the *immediate* lexical container — the enclosing step, facet, or workflow — and `$.attr` reads that container's parameters *and* its return fields. `$$` names the container one level out, `$$$` the one beyond that, and walking past the outermost container is a compile error, not a silent `null`. A block sees exactly two things: the `$…` attributes of its enclosing containers, and the steps declared in its *own* block. It cannot see a sibling block's steps, and — this is the part that surprised me — it cannot name the step that contains it. The containing step is reached only through `$`. `andThen when` obeys the same rule: you attach the guard to the step it gates and resolve names from there.

Now, the up-level operators, which you are right to be suspicious of. I resisted them. I wanted `$` and nothing else. The example that forced my hand is nested step bodies that each declare a same-named parameter:

```
o1 = One(input = $.input) andThen {
  o2 = One(input = $.input + $$.input) andThen {
    o3 = One(input = $.input + $$.input + $$$.input) andThen { … }
  }
}
```

Here `o3` legitimately needs three different `input`s: its own container's, `o2`'s, and the workflow's. Under the flat model those three names collided and only the outermost was reachable. Under the relative model each is spelled unambiguously — `$.input`, `$$.input`, `$$$.input` — and there is *no other spelling available*, because, as I said, the containing step is not nameable. So the up-level operator is not noise I added for symmetry; it is the minimum mechanism that makes a legitimate program expressible at all. I would rather have a slightly Perl-ish sigil that can say the thing than a clean syntax that cannot.

There is a deeper version of this idea that I considered and deliberately parked, and I want it on the record because it is a road I chose not to take. The relative operators walk *lexical* containers. One could imagine `$$` in a facet body reaching not its lexical parent but its *caller* — the container that instantiates the facet, as though the facet's body were inlined at each call site, with the compiler checking at every use-site that the instantiating container actually provides the named field. That would make a facet a kind of open term parameterised by its context. It is expressible and it is checkable. I parked it for one reason: every concrete case I found where I wanted it was better served by an explicit parameter. A facet that reaches into its caller's scope is a facet whose contract is invisible at the call site, and invisible contracts are precisely what the DSL exists to prevent. So the up-level operators are lexical only, and the caller-reaching version stays in the drawer with a note explaining why.

Q8b (Dr. Yegge)

> *Two practical things. First: relative nesting sounds like it pushes authors toward deep pyramids — if I have to gate one step on five earlier ones, do I end up five `$` deep? Second: you changed the scope rules of a language that already had live workflows running against it. How did you do that without a flag day?*

On the first: yes, naive nesting pushes you toward pyramids, and I hit exactly the failure you are describing. I had a workflow where a final step needed to see five prior results, and the obvious relative-scoping transcription nested them five deep — a `$$$$$` tower — which was unreadable, and worse, it *serialised* five steps that had no data dependency on each other simply to stack them in one another's scope. The nesting bought visibility at the cost of parallelism, which is a bad trade.

The idiom that fixes it is decomposition, not deeper nesting: pass the prior steps as *facet-typed parameters* to a small gating facet. The five steps stay siblings in one block — parallel, no artificial ordering — and the gating facet receives them as typed inputs and refers to each as `$.param.field`. Every reference is one level deep. The facet is reusable, and its signature documents exactly what it gates on, which the pyramid never did. The nicest property is that this idiom is model-agnostic: it validates identically under the old flat scoping and the new relative scoping, because it never reaches across scopes at all — it passes data explicitly. When I found a construction that was clean under *both* models, that was a strong signal it was the right one, and it is now the pattern I steer authors toward the moment a pyramid starts to grow.

On the second — the live migration — this is the part I am most pleased with operationally, because we did it without a flag day. The relative runtime was built to still run legacy flat FFL unchanged: at top level `$.x` still means a workflow input, step references still resolve the same way, and there is an injected-inputs fallback for the constructs that relied on the flat container. So the two models are not a hard fork; the new runtime is a strict superset of the old behaviour on old programs. That let us ship it flag-gated — dual-model behind `FW_FFL_RELATIVE_SCOPING` — run both interpretations side by side, provide a scope-aware migration tool that rewrote workflows into the relative idiom, flip the default once the corpus was migrated and green, and only then remove the old path. A live language migration on a running system, staged rather than switched.

One implementation note the committee touched on earlier. The cross-block deferral mechanism — the `_StepNotReady` signal I raise when a block references a step that has not produced its value yet — was not deleted in the rework. It was *repurposed*: it is now what resolves `$.field` references to a container's *return* fields, which are not available until the container's body has run. The same machinery that used to paper over cross-block ordering now does honest work resolving relative container-return references. I mention it because it is a case where a mechanism I might have been tempted to rip out during a scope-model change turned out to be exactly the primitive the new model needed.

Q9 (Prof. Pike)

> *No recursion. A workflow author who needs to walk a tree — a dependency graph, a hierarchical resource — cannot express that in FFL. You call this "a deliberate restriction that makes topology statically tractable." What do you say to the author who has a recursive problem?*

That they have two options and neither is wholly satisfying.

Option one: unroll the recursion at workflow generation time. If the tree depth is bounded by a compile-time constant or a runtime parameter known at workflow invocation, the workflow can be generated by an external templater before submission. This is the CWL approach for similar cases. It is ugly and it loses the language-level topology check below the generation layer.

Option two: implement the recursion inside a single handler. A facet named `WalkTree` takes a root and internally walks the tree, returning an aggregated result. This moves the recursive structure into handler code, where it is not visible to the workflow and not statically checkable, but it *is* expressible and is sometimes the right answer.

I should be careful about how I state what is missing, because an earlier draft over-generalised it to "dynamic workflow expansion at runtime based on intermediate results", and that is not a gap — FFL does exactly this, in production. A step returns a collection discovered at run time and a `foreach` fans out over it:

```
children = county.atlas.ListCounties(prefix = $.prefix, bucket = $.bucket)
    andThen foreach cty in $.counties limit $$.concurrency { … }
```

The fan-out width is unknown at compile time; the 3,167-way national run of §14 was determined entirely by what that step returned. Expansion driven by an intermediate result is ordinary FFL.

What is genuinely absent is **unbounded recursive** expansion: a step whose output spawns another level of the same structure, repeatedly, to a depth not known when the run starts. That is the web-crawler and tree-walk case, and it is a real gap. Authors who need it should use a different system, or wait for the cross-workflow composition primitives in §14.2: once a workflow can call another workflow as an event facet with runtime-determined parameters, recursive walks become expressible as mutual workflow calls, which the runtime's actor-model block evaluator will serialise correctly.

I do not think FFL is universally adequate. I think it is adequate for the workloads I target. A recursive web crawler is not one of those workloads.

III. The coordination protocol

Q10 (Prof. Gray)

> *Your entire coordination story depends on a MongoDB-specific feature set — single-document atomicity, partial unique indices, lease expiry via queries. A team that standardises on PostgreSQL or another store is excluded. Is this a reasonable architectural commitment?*

It is a reasonable commitment for the workloads I target. It is an unreasonable one if Facetwork claims to be universally deployable, which it does not.

PostgreSQL can implement the claim protocol via `SELECT ... FOR UPDATE SKIP LOCKED` with a compound unique constraint, and I believe a PostgreSQL port of Facetwork is about four weeks of work. The atomicity semantics are different but the protocol shape is identical: atomic read-modify-write, filter by state, transition to the new state. The partial unique index becomes a conventional unique index with a nullable discriminator.

CockroachDB, FoundationDB, and YugabyteDB all support the required primitives, each with their own subtleties. DynamoDB does via conditional writes. Kafka alone is awkward, because the claim model is pull-based and Kafka's consumer-group protocol is semantically distinct, but with Kafka Streams' state stores one can approximate it.

What would be lost in a PostgreSQL port is the operational knowledge teams have built up around MongoDB's specific failure modes (step-down behaviour during primary rotation, election delays, etc.). For teams already running PostgreSQL, those costs are paid in PostgreSQL's own failure modes, which are well-understood in that community.

I will accept the criticism that the thesis should state more prominently that the claim protocol is portable, and that MongoDB is an incidental rather than essential choice. It is in Chapter 15 but it deserves to be in Chapter 5.

Q11 (Dr. Yegge)

> *Operationally — let us say I run Facetwork in production. I have twenty-five handlers in five languages spread across a thousand nodes. A new handler is registered, and I discover a bug in it two hours later. What actually happens?*

Four sequential actions, each observable in the dashboard:

1. I push a fix to the handler module, to whatever code path the language requires — for Python, to the modules directory; for Scala, to the packaged jar; for the language in question, the appropriate location.
2. I call `register_handler(...)` with the updated registration metadata — possibly identical to the previous registration if only the code changed, or with new timeout or parameter metadata if those changed.
3. I run `fw fleet rolling-deploy` across the fleet, which quarantines each runner in turn, waits for its in-flight tasks to complete, stops it, restarts it with the new code, and un-quarantines it. Tasks in flight at the moment of bug detection that were executing the buggy handler will complete with the buggy behaviour; tasks not yet claimed will be claimed by runners with the fixed code.
4. For tasks that completed with the buggy behaviour before the deploy, I use the dashboard's **Re-run From Here** action to reset each affected step and re-execute downstream from it. The run history reflects the re-run with a step log entry recording the operator and time.

The total fleet-wide throughput drops during the rolling deploy but never reaches zero. The dashboard renders the deploy as a time-sequence of per-runner lifecycle events so that an operator can watch the wave propagate. Customer-facing latency for new workflows is unaffected because there is always a subset of runners available to claim new work.

The scenario that is harder — and I will be honest — is a bug in the handler code that corrupts downstream state in a way that is not visible from step-level re-runs. For those, you need human investigation, and no workflow system on earth will save you from that need.

Q12 (Prof. Kleppmann)

> *The actor-model block evaluator described in §5.7. You claim redundant "continue" messages become idempotent no-ops. But an idempotent no-op still consumes a task claim, a poll cycle, a database query. At scale this is not free. Quantify it.*

At the scale I have evaluated (three runners, workflows with hundreds of steps), the redundant continue messages amount to a few percent of total claim overhead. In absolute terms this is negligible.

At one thousand runners with blocks of high fan-in (say, ten prerequisites completing within a heartbeat interval), each block produces on the order of ten continue messages of which nine are no-ops. The cost is ten claim operations per block evaluation instead of one. If blocks evaluate at, say, a thousand per second across the fleet, that is ten thousand extra claim operations per second — a non-trivial load on the MongoDB primary.

Two mitigations are available. First, continue messages can be coalesced at the emitting runner: if runner R has multiple completions that all target the same block, it emits a single continue message rather than one per completion. This is a small patch and the correctness argument is straightforward (the coalesced message carries no information beyond "some completions happened," which is what the evaluator reads for itself anyway). Second, the evaluator's no-op detection can fast-path: if it reads a block's state and nothing is newly ready, it can complete without writing anything, reducing the write cost to a single read.

I have not implemented either optimisation because the current scale does not require them. In production at ten thousand concurrent workflows, either or both would be necessary. The design is correct as stated; the performance optimisations are straightforward.

Q13 (Prof. Gray)

> *Your reaper that reclaims expired leases — you show a `find_one_and_update` with `lease_expires < now`. If two runners race on this for the same expired document, are you certain only one succeeds?*

Yes, and the argument is worth stating precisely.

`find_one_and_update` is atomic on the document. The two racing runners each issue the same conditional update: match `state = "running"` and `lease_expires < now`. The first to arrive at the document sees a match and atomically sets a new `lease_expires = now + lease_ms`, along with its own `server_id`. The second runner's query arrives moments later, but by the document's new state, `lease_expires > now`, so the second runner's filter does not match, and the operation is a no-op that returns no document. The second runner receives `None` and moves on to try another task.

This is not a subtle property. It is the defining property of atomic compare-and-swap, which `find_one_and_update` implements at the document level. The atomicity is guaranteed by MongoDB's storage engine regardless of replica state, because the operation is scoped to a single document on a single replica (the primary) at one moment in time. If, hypothetically, both runners were aimed at different replicas — a misconfiguration we do not support — the replicas would diverge momentarily and resolve through the replica-set's own consensus protocol; but that is not the supported topology.

Q14 (Prof. Abadi)

> *Let us stress-test the safety argument. MongoDB undergoes a primary step-down mid-claim. The old primary has just written* `state = "running"` *and committed to its local oplog but has not replicated to the new primary. What does your claim protocol see?*

Two cases.

Case A: the write was acknowledged with majority write concern (which Facetwork's configuration requires). Then by MongoDB's majority-acknowledgement semantics, the write is in the oplog of at least a majority of replicas before the client receives acknowledgement. A new primary elected from that majority sees the write. Replicas that do not see the write roll back their local state during the election. The claim remains valid.

Case B: the write was acknowledged with less-than-majority write concern. Then the new primary may not see the write, and the task will appear to be pending to the next claimant. The old primary's claim is lost, and the old primary's handler — if it was still executing — is now a zombie with respect to the new primary. The safety property on the task document is preserved (no two `state = "running"` documents exist at any instant), but the old handler's external side effects may double with the new claimant's.

Facetwork's configuration uses majority write concern. I consider sub-majority write concern on the task collection to be a configuration error, and the framework should reject it at startup. I did not add that check because I did not anticipate the objection; I will add it.

IV. Recovery and determinism

Q15 (Prof. Liskov)

> *Your argument against Temporal's determinism constraint describes it as a "tax" that rules out non-deterministic workloads. But a properly-factored Temporal workflow confines non-determinism to activities, which Temporal explicitly treats as non-deterministic. You are arguing against a caricature.*

Partially conceded. Let me restate where I think the difference is real and where it is caricature.

It is caricature to say that Temporal workflows cannot contain non-determinism. They can, via activities. A well-factored Temporal workflow places essentially all non-deterministic operations in activities, and the replay machinery handles them transparently.

It is not caricature to say that the line between "code that belongs in the workflow" and "code that belongs in an activity" is not the line that a domain expert would naturally draw. A scientist writing a bioinformatics pipeline does not think in terms of workflow code and activity code; they think in terms of pipeline stages. Temporal forces the author to partition their logic twice — once into pipeline stages, once into workflow vs. activity — and the second partition is an artefact of the replay model, not of the problem.

Facetwork requires the first partition (into facets) and does not require the second. For workloads where the second partition is easy (payment flows with a small number of well-delineated external calls), Temporal's model is elegant. For workloads where the second partition is difficult (data pipelines with many small decisions informed by runtime data), Facetwork's model is less ceremonious.

I accept that the thesis should be clearer that the determinism "tax" is a cost-at-authorship-time, not an outright prohibition. I maintain that for the target workloads it is a real cost and that authors regularly pay it as "workflow-activity bleed" refactors that appear later.

Q16 (Prof. Kleppmann)

> *Step-level re-run is dangerous. If a completed step sent a Slack message, and the operator hits "Re-run From Here" on that step, Slack gets the message twice. You have no story for this. Either admit it or explain.*

Admitted, and I will elaborate.

Re-run From Here deletes downstream steps and re-executes the chosen step with its original inputs. Any external side effect in that step is, by default, repeated. For handlers that are not idempotent, this is unsafe.

The mitigation is the same as for the zombie-handler problem: external idempotency. If the handler's external call uses a deterministic idempotency key derived from the task's UUID, the external system de-duplicates. This is standard practice for payment APIs, message queues with idempotent producer support, and many SaaS APIs. Slack, infamously, does not offer idempotency keys on message posts, so for Slack specifically the mitigation is to either (a) tag the handler with a `@not_retryable` annotation that causes Re-run From Here to ask for operator confirmation with an explicit warning, or (b) move the Slack post into a separate "notification" handler chained to the main work so that re-running the main work does not re-post.

The broader admission is that Facetwork's recovery model puts the burden of idempotence on the handler author, and the tooling to make this easy — annotations, warnings, framework-level idempotency-key plumbing — is not yet built. Temporal's model puts that burden on the replay infrastructure, at the cost I discussed in Q15. Neither system solves the problem for handlers that interact with external APIs without idempotency support; both systems give the author a path to safety. Facetwork's path is less paved; Temporal's is more so. I should say this more plainly in Chapter 15.

Q17 (Prof. Gray)

> *Schema evolution. A handler adds a new field to its return schema. In-flight workflows that were serialised with the old schema now fail validation. Facetwork has no versioning. This is inadequate for enterprise use.*

Three responses, graded by how much I concede.

First, most schema changes are *additive*: a new field with a default value. Facetwork's type checker is structural on record literals — the `{ name: String, size: Int }` required by a downstream facet accepts a value with extra fields. So additive changes do not break in-flight workflows; they simply add fields that downstream facets ignore.

Second, non-additive changes — renames, type changes, removals — require explicit migration. Facetwork does not automate this. The operator's options are: (a) finish all in-flight workflows under the old schema before deploying the new one (practical for short workflows, impractical for multi-day ones); (b) write a migration handler that transforms old persisted state to the new schema and run it before the new handler is activated; (c) version the facet by name (`MyFacet.v2`) and deprecate the old version slowly.

Third, I concede that Facetwork has no *declarative* schema versioning, no automatic migration runner, no compiled compatibility check. For enterprise use with long-running workflows, this is a genuine gap. The roadmap in §14.2 flags this as "typed schema migrations as first-class FFL constructs," and I believe the design for this is a dissertation's worth of work on its own — analogous to Protobuf's evolution rules but for workflow state. I am not going to fake having solved it.

V. Comparisons

Q18 (Prof. Liskov)

> *You are unfair to BPMN in Chapter 10. BPMN interchange is a feature for enterprises with regulatory obligations. It is not mere "weight."*

Partially accepted.

I am unfair in the specific sense that I equate BPMN's verbosity with its cost, while its verbosity is partly a consequence of its being a standard that must specify semantics precisely across tool boundaries. A BPMN diagram can be exported from Camunda and imported into Flowable, and this is valuable for organisations that cannot tolerate tool lock-in.

I am not unfair when I say that BPMN's canonical form (XML) is poorly suited to version control, code review, and textual editing. These are facts about the serialisation, and they are not changed by the interchange-standard argument.

Where the criticism is strongest: for organisations whose workflow *is* their regulatory artefact — auditable process flows in financial services, healthcare, compliance contexts — BPMN's standardisation pays off in ways an unstandardised DSL cannot. Facetwork is not the right choice for those organisations. My thesis should be more explicit that the target workload class *excludes* workflows whose notation itself is an artefact of a regulatory regime.

I accept the criticism to that extent. I do not retract the broader point that for engineering teams building long-running data workflows, BPMN's weight is felt daily and its interchange benefits are rarely exercised.

Q19 (Dr. Yegge)

> *Temporal Cloud is a commercial product with SLAs, a support team, and years of production validation. Facetwork is a research prototype. Would you deploy it in a bank?*

Not today. Not because the design is wrong, but because the maturity is insufficient.

The design could run in a bank, subject to three kinds of maturation. First, the operational tooling — for backup, restore, migration, cross-datacentre replication — would need to be built out to the standard a bank's operations team expects. MongoDB provides the underlying replication; Facetwork provides the workflow-level tooling on top, and much of that is not yet at bank-grade. Second, the idempotency and schema-migration gaps discussed above would need to be closed — or the bank's workflows would need to be written with those gaps in mind, which is a non-trivial authoring constraint. Third, the documentation, the training, the certification path, the 24/7 support — none of these exist for Facetwork, and a bank will not adopt a system without them.

Temporal Cloud has invested substantially in the second and third. A realistic answer is: a bank that wanted Facetwork's design properties would need to either pay for equivalent support — from the author, from a consultancy, from a new commercial entity — or accept the operational cost of building the tooling in-house.

My thesis is about the design, not the commercial readiness. The commercial readiness is another question, and it has a business answer rather than a technical one.

Q20 (Prof. Kleppmann)

> *Your Jenkins contrast in Chapter 12 is a straw man. Nobody serious builds long-running data workflows in Jenkins. You are knocking over a tool nobody is using for your target workload.*

True, and I acknowledge it in §12.4. The contrast is there because Jenkins is widely familiar and represents a specific architectural shape — centralised controller, imperative pipelines, tight coupling of scheduling and execution — that many engineers have intuitions about. Reading Chapter 12 as "why Facetwork is better than Jenkins for long data pipelines" is strawmanning. Reading it as "these architectural choices that you know from Jenkins are ones Facetwork deliberately avoids" is the intended framing.

I will concede that the chapter's title could be less confrontational. The content is meant to be about architectural inheritance, not about CI/CD competition.

Q21 (Prof. Pike)

> *Temporal, Camunda, Airflow, Jenkins. You cherry-picked comparisons. What about Argo Workflows, Prefect, Dagster, Flyte, or Conductor? These are closer competitors than Jenkins.*

Concession: the thesis should compare against Argo and Prefect directly and does not. Let me offer shorter answers.

Argo Workflows is YAML-on-Kubernetes. Its strengths are Kubernetes-native scheduling and the operational benefits of container isolation. Its weaknesses are the static nature of YAML DAGs, the per-step pod-startup cost (seconds minimum), and the limited expressiveness compared with FFL's `andThen when` and mixins. For workflows dominated by container startup latency, Argo is penalised; for workflows where every step is a long-running process, Argo is competitive.

An earlier draft of this answer claimed Facetwork's per-task overhead is "in the tens of milliseconds rather than seconds", and used that to claim an advantage here. **That is wrong and I withdraw it.** §13.3 measures a Facetwork step at roughly 2.6 seconds of engine overhead — a bootstrap task, a persisted step row, a task row, a claim poll — with the step-state cascade advancing at about nine steps per second even after two rounds of optimisation. Snakemake dispatches at 14 ms per task and Nextflow at 35 ms. Facetwork is two orders of magnitude off the figure I quoted, and it is in the same *seconds* regime as the pod startup I was contrasting it against. The honest comparison with Argo is therefore not "we are faster per step"; it is that we buy durable, queryable, resumable per-step state for a comparable per-step cost, and that neither system is the right choice for thousands of millisecond-scale tasks (§2.6 argues for delegating those to a compute substrate instead).

Prefect and **Dagster** are Python-library workflow runtimes with DAG semantics and code-based authoring. They share Airflow's weaknesses at the scheduling layer (historically single-scheduler, though both have improved) and Airflow's strengths at library ecosystem. Against Facetwork, their main weakness is the same as Airflow's: no typed language-level separation of topology from implementation, and no live-updatable multi-language handlers.

Flyte and **Conductor** are closer to Facetwork in architectural shape — typed workflow definitions, persisted state, multi-language support. Flyte in particular has a strong type system for data-science workflows. A proper comparison chapter would compare against Flyte in detail. I have not done it because the thesis was already long and Flyte's popularity is regional; but the committee is right that the thesis is incomplete without it.

I will add a Flyte comparison before the final submission.

VI. Evaluation

Q22 (Prof. Kleppmann)

> *Your evaluation in Chapter 14 is one workload. No benchmark, no throughput measurement, no comparison, no statistical treatment. This is an existence proof, not an evaluation.*

Accepted, with clarification.

The OSM geocoder is an existence proof that the design choices hold together on a realistic long-running workload. It is not a benchmark showing that Facetwork is faster or cheaper than Temporal on the same workload — which would require implementing the same workload in Temporal, running both under controlled conditions, and measuring.

I did not do this for three reasons, two honest and one slightly defensive. First, implementing a multi-hour OSM import twice is a large amount of work and was not the thesis's contribution. Second, performance comparisons between workflow systems are genuinely hard to design well; the systems make different assumptions and a benchmark that favours one over the other can often be devised. Third, defensively, the thesis argues for a design that serves a class of workloads; performance is one dimension of that service and not the most important one.

I accept that the thesis is weaker without a controlled head-to-head against Temporal on the same workload, and that a committee member who requires one can reasonably withhold approval pending it.

I should update this answer, though, because it was written before the measurement that does now exist, and leaving it as drafted understates the thesis. Chapter 13 and `experiments/` contain the same workload implemented in **Facetwork, Snakemake and Nextflow**, with all three calling identical handler code so that what differs between the runs is only what schedules them, and with their outputs diffed to confirm they agree byte for byte. That covers a two-step pipeline and a 50-way fan-out, at 200/800/3,167 width, and it produced results I did not expect and would not have invented — a Facetwork fan-out paced by its recovery sweep rather than by execution, and Snakemake serving a stale answer after an algorithm change.

What exists now:

- **Cross-engine, same workload, verified-identical output** — §13.3 and the two experiments.
- **Fan-out scaling and its cost decomposition** — execution versus cascade separated at three widths, which is what located both runtime defects (844.6s → 55.8s, and 294.1s → 4.4s on a real workflow).
- **Persistence profiling with call-site attribution** — 17,011 store calls for a 157-step workflow, which disproved my own hypothesis about where the cost was.
- **Timeout interactions** — `paper-timeout-interactions.md`, which is the staged-versus-flat comparison this answer promised.
- **Environment provisioning** — `paper-environment-provisioning.md`, including results where Facetwork's lazy tier *loses*.

What is still missing, and I will not pretend otherwise: a microbenchmark of the claim protocol's throughput in isolation, a latency measurement of step-level recovery versus workflow-level restart, and the Temporal head-to-head. The measurements above are also single-host, taken on a workstation with other runners live, and should be read as engineering measurement rather than as controlled benchmarking.

So the fair statement is no longer "this thesis is design without measurement". It is that the measurement is of *this* system's behaviour, and comparative against two systems in the scientific-pipeline tradition, but not yet against the durable-execution systems that are its nearest architectural neighbours.

Q23 (Dr. Yegge)

> *The OSM evaluation mentions an OOM kill on one runner as a success case. You lost three hours of work. A Temporal equivalent would have replayed the history and resumed at the exact point. Your "success" is Temporal's "failure mode."*

Not quite. Let me be precise about what each system would actually do.

For a Temporal activity that takes three hours to complete, an activity timeout is set to some value — typically larger than the expected duration with margin. If the worker dies mid-activity, Temporal reclaims the activity on another worker and re-runs it from the start. The activity restarts from zero, just as in Facetwork. Temporal replay handles the *workflow code's* state (which decisions have been made, which activities have been called); it does not resume a partially-executed activity.

So both systems lose the three hours of re-work when a runner dies mid-activity. Where Temporal wins is: the workflow *as a whole* does not need to be re-orchestrated. Facetwork also does not re-orchestrate the whole workflow — the workflow state is in MongoDB and survives the runner death — so the two systems are equivalent at this level too.

Where Facetwork loses relative to a hypothetical Temporal with *checkpointed activities*: neither system has this in practice. Checkpointed activities are an active research topic; Temporal has some experimental support. Facetwork's handler authors can implement their own checkpointing — writing intermediate state to the task's `progress_pct` and reading it back on retry — but the framework does not help them.

I concede that Temporal's activity model provides cleaner semantics for "the workflow code survives, the activity re-runs." Facetwork provides equivalent semantics with different naming ("the workflow state survives, the handler re-runs"). Neither system automatically resumes a partially-executed long activity, and claiming otherwise for either would be misleading.

Q24 (Prof. Abadi)

> *Your staged-timeout mechanism is claimed as a contribution. Demonstrate experimentally that it changes observable behaviour in a way nothing else does.*

The demonstration is this: run the OSM import of a large region (France, 4 GB PBF) under three configurations and observe which complete.

Configuration A: global timeout = 15 minutes (Facetwork's default), no staged timeouts. Observation: handler is killed during the PBF scan at the 15-minute mark, task is reset to pending, next claim re-starts the scan, handler is killed again. Infinite retry loop until max_retries exhausts and the task dead-letters.

Configuration B: global timeout = 48 hours, no staged timeouts. Observation: handler completes successfully. But a genuinely stuck handler — one where the code has deadlocked in a C extension and heartbeats have stopped — would also run for up to 48 hours before being noticed, losing 48 hours of fleet capacity.

Configuration C: global timeout = 4 hours, staged timeouts active (PBF scan budget scaled from file size). Observation: handler completes successfully. A genuinely stuck handler is killed at 4 hours by the global timeout. The stage budget extends the deadline only during declared legitimate stages; idle deadlock is caught.

The claim is that configuration C achieves what neither A nor B can. A is too short; B disables the watchdog; only C distinguishes legitimate long stages from stuck handlers. I observed exactly this behaviour during the OSM evaluation: configuration A was the original deployment, configuration B was the interim response, configuration C is the state after the work in Chapter 8.

This is not a benchmark of throughput. It is a demonstration of a qualitative capability. I will add it to Chapter 14.

VII. Scope, future work, and impact

Q25 (Dr. Yegge)

> *You describe your target workload as "live long-running domain workflows." Quantify: how many engineering teams, worldwide, have this workload? Is your work relevant to anyone?*

I do not know the number. I can give you a rough lower bound from the domains I have concrete evidence for.

Bioinformatics pipelines: every major genomics institute and every clinical sequencing lab. Nextflow and Snakemake have tens of thousands of users each. Facetwork's target is the subset of these who want live updatability and step-level recovery, which I estimate in the low thousands.

GIS and Earth-observation data processing: hundreds of organisations, from governmental mapping agencies to commercial satellite-imagery providers. Each runs multi-hour to multi-day ingestion pipelines that currently use Airflow, custom Python, or commercial ETL tools.

Drug discovery simulation, materials science, physics simulation — each has workflow-shaped workloads and each suffers from the tools they currently use, which are either Airflow-shaped (centralised scheduler, brittle recovery) or ad-hoc shell scripts.

ML pipelines with model-training and evaluation stages — every major ML organisation has this workload, and the current tooling (Airflow, Kubeflow Pipelines, Metaflow) has the limitations I describe.

Cumulatively, the addressable user base is in the low-to-mid thousands of engineering teams. That is modest compared to Jenkins (millions) but comparable to Temporal's deployed user base. Facetwork is not aimed at mass adoption; it is aimed at teams whose specific pain points match the design choices.

Whether the work is *relevant* to anyone depends on whether the design properties — DSL authorship, live updatability, step-level recovery, staged timeouts — matter to those teams more than the operational maturity costs they would incur to adopt a research-grade system. For most teams today, that balance tips toward not-adopting. The thesis argues for the design; the adoption argument is a business one.

Q26 (Prof. Liskov)

> *Three concrete failure modes in production that you have not addressed in the thesis.*

Three, given freely.

First, MongoDB primary step-down during a high-velocity claim loop. The step-down typically takes 10–30 seconds. During that window, claims fail. Runners retry with backoff and eventually succeed on the new primary. But during the window, stage-budgeted long tasks may time out if their lease renewal fails to reach the new primary. I have not stress-tested this; the design should be robust but the operational behaviour under step-down is not documented.

Second, partial network partitions between handlers and MongoDB. A handler that can *read* MongoDB but cannot *write* — asymmetric partition, or a slow write path — heartbeats locally but its writes are queued in the driver's buffer. Its lease expires, another runner reclaims, and when the partition heals the original handler's delayed writes land. The document-level atomicity prevents the original handler's state-write from being accepted (the task is no longer in the state it expects), but its progress writes and step log writes land spuriously. The dashboard shows apparent progress from a handler no longer holding the task. This is a minor cosmetic issue rather than a correctness one, but it is real.

Third, handler memory leaks in the RegistryRunner model. The Python RegistryRunner loads handler modules dynamically and keeps them resident. A handler with a small per-invocation memory leak accumulates over hundreds of tasks. The runner's memory grows over days, and eventually it OOM-kills. There is no framework-level mitigation. Operators periodically restart runners as a hygiene measure. A principled solution would be per-task process isolation, which is what AgentPoller offers, but at the cost of startup overhead. I have not designed a middle ground.

Each of these is a real production concern. None is a design flaw; each is an operational concern the thesis under-weights. I will add a Chapter 15 subsection titled "Production concerns the design under-weights" that names them.

Q27 (Dr. Yegge)

> *If you had to start over, what would you change?*

Two things.

First, I would design the handler context API from the beginning with the richer capabilities — stage budgets, idempotency keys, structured logging — rather than growing them as flat payload keys and then wrapping them in a context object. The current dual API (flat keys for backward compatibility, context object for new code) is a cost I pay every time a handler author must decide which to use.

Second, I would design the workflow repair mechanism as a general *state reconciliation* system rather than a bespoke tool. The current repair script is a fixed sequence of five checks. A more principled approach would be to express the invariants the system maintains (for each step, some state transitions are legal; inconsistent states are detectable) and have a reconciliation engine that brings the system back into a consistent state from any reachable inconsistent state. This is Kubernetes-controller-like design, and I think it would serve Facetwork well, but I did not have the insight when I started.

I would not change the DSL. I would not change the coordination protocol. I would not change the state-recovery model. The three major design choices have aged well in my experience and I stand by them.

Q28 (Prof. Pike)

> *Summarise in ninety seconds why your thesis matters.*

Workflow systems are among the most widely-deployed pieces of infrastructure in modern software. Every team that does data processing, ML training, scientific computing, or multi-stage release engineering interacts with one. Yet the existing systems force trade-offs — between expressiveness and static checking, between central control and fault tolerance, between determinism and realism — that make them awkward for an important class of workload: long-running, heterogeneous, domain-authored workflows maintained by teams that change their handlers frequently.

The thesis argues that a different set of design choices — a typed DSL, document-atomic coordination, state-persisted recovery, live operational control — resolves the trade-offs better for that workload class. The argument is both constructive (Facetwork exists and runs) and analytical (the choices reinforce rather than undercut each other, which is not obvious and requires argument).

If the argument is right, the lesson is not that every workflow system should adopt these choices — different workloads want different trade-offs — but that the design space has a richer structure than the current market suggests. A team building a new system today has choices better than "Python with decorators plus a coordinator," and the specific combination I have defended is worth knowing about.

If the argument is wrong, the specific combination is still interesting as an existence proof: it shows that a typed DSL can coexist with document-atomic coordination, that lock-free work claim can support a live-updatable fleet, and that stage budgets can compose with global timeouts. Each of those is, in small, a contribution.

That is ninety seconds. Thank you for the time.

Candidate's closing statement

I want to thank the committee for the questions. Several of them — the zombie-handler result-write gating, the MongoDB write-concern startup check, the Flyte comparison chapter, the idempotency-annotation framework, the stress-test configurations for staged timeouts — I will act on before the final submission. Several others — the determinism-tax caricature, the Jenkins-strawman framing, the BPMN-interchange concession — I will soften in the revised text.

The questions about evaluation depth are the ones I am most conscious of. This thesis leans *design*, but less than the earlier draft of these answers admitted: Chapter 13 now carries the same workload implemented in three engines with outputs verified identical, fan-out cost decomposed at three widths, and two runtime defects found and fixed by measurement rather than by reading. What it still lacks is a controlled comparison against a durable-execution peer — Temporal — and isolated microbenchmarks of the claim protocol. If the committee's judgement is that those are required, I will do them. If the judgement is that the design argument stands and they are separable follow-on work, I will accept that too.

I do not expect the committee to agree with every argument in the thesis. I do hope to have made a case that the design is coherent, the choices reinforce each other, and the class of workloads addressed is worth addressing.

Thank you.

Addendum (October 2026): answers that no longer stand, and the questions I would now expect

Six months of operation since the defense have falsified parts of three answers and raised questions the committee did not ask. I record them here rather than editing the transcript, so the original answers remain visible beside their corrections. The thesis carries the details in §13.4c and §15.10–§15.14.

Corrections

Q14 (write concern). I said Facetwork's configuration requires majority write concern. It does not, and could not have: the evaluated deployment is a single standalone MongoDB node with no replica set. Nothing in the code sets a write concern. The step-down scenario Prof. Abadi posed therefore does not arise. The honest answer is the less comfortable one: losing that node stops the fleet, and nothing replicates its state. The start-up check I promised has since been added, in a different form from the one I described. On a replica set or sharded cluster it refuses a write concern below majority, reading the server's default when the client sets none. On every topology it refuses unacknowledged writes (`w=0`). On a standalone node there is nothing to check, so it reports the single point of failure, and refuses it only when `FW_REQUIRE_REPLICATED` is set. A default it cannot read is reported as unverified, not as safe (`facetwork/runtime/mongo_store/write_concern.py`; also shown by `fw db check`). Run against the fleet, it reports a standalone node, which is the honest answer.

Q12 (redundant continue messages). I said coalescing was unnecessary at the scale evaluated. It became necessary at a 255-way fan-out, where un-coalesced continue messages made block evaluation $O(N^2)$ and stalled the run. It is implemented (thesis §5.7.5, §15.5). The larger load came from a source I did not consider at all: polling at idle. With 37 runners doing nothing, the fleet issued 190 claim operations per second, all of them empty. Back-off in the agent protocol and one claimer per host brought that to 2.1 per second (§15.11).

Q11 (deploying a handler fix). The path I described — `register_handler` followed by `fw fleet rolling-deploy` — still exists, but it is no longer how the fleet deploys. Domain handlers are now baked into a multi-architecture runner image, and a fix ships as an image rollout (`fw fleet rollout`): each host's agent reconciles to the new image, staggered across hosts. "Re-run From Here" remains the remedy for work the buggy version completed.

The closing statement's undertakings.

- *Done:* result-write gating for zombie handlers (`save_task_if_owned`), and effect annotations (`with Effect(kind = …)`), which tell a reader whether repeating a facet is safe.
- *Done late:* the write-concern check (above), added after this addendum was first written.
- *Not done:* the Flyte chapter.
- *Done differently:* the timeout work became a paper on expiry ordering (`paper-timeout-interactions.md`) rather than a set of stress-test configurations.

Questions I would now expect

Q29 (Prof. Kleppmann)

> *Your abstract says exactly-once. Does a task's work ever run twice?*

Yes, and the abstract was wrong; it has been corrected. The protocol guarantees one claimant at a time, not one execution. The window that matters is the dead-server reaper, not lease expiry. A runner that stops heartbeating for two minutes is presumed dead, but a paused or hibernating host keeps executing, and there is no fencing token on what it writes to the object store. On this fleet that is routine: one host hibernated for 7.4 hours in the middle of a run. I have also measured a second path. In 11 of 666 event tasks inside capped fan-outs, a second task was created milliseconds after the first completed. A commit-time guard now refuses it, though I could not reconstruct the stale read that produced it. The contract is at-least-once, and handler idempotency is a requirement written into the documentation, not a hope.

Q30 (Prof. Gray)

> *You now run a claimer process on every host that claims for every runner there. Is that not the coordinator you spent Chapter 5 arguing against?*

It is a coordinator for one machine and nothing more, and the distinction is load-bearing.

- Two claimers race on the same atomic claim, exactly as two runners did.
- A claimer holds nothing durable. A task it hands over is written with the same ownership-gated update, and a runner that cannot reach its claimer claims directly.
- Removing every claimer returns the fleet to the protocol of §5.3 unchanged.

What it adds is a place to *reserve*, which a stateless claim cannot do. That turned out to be its most important job (Q31).

Q31 (Prof. Abadi)

> *Your liveness argument assumes that if a task is pending, some live runner matching it will claim it. Does it still hold?*

Only for satisfiable work. A task can now carry a resource floor and an environment, and both can leave it unclaimable without any fault occurring. The measured case: a 13 GB build waited 111 minutes with nothing running anywhere, because every host had the capacity and none had it free; idle runners held it. The check meant to catch this compared floors with *capacity* and reported health. It now compares them with the free memory runners report, and calls such a task starved. The opposite failure followed. A floor that is checked but not reserved let one host claim three 11–12 GB builds in the same second. The host claimer now keeps a reservation ledger, and its first live test found a claim path that ignored floors altogether. I would restate the liveness property as: progress for any task some live runner can satisfy *now*. Satisfiability is checked, not assumed.

Q32 (Prof. Liskov)

> *You have five implementations of the agent protocol. How do you know they agree?*

Until October 2026 I did not, and they did not. All four non-Python agents claimed with no lease and no retry limit, so a failing handler in Java retried without bound, where the same handler in Python would have dead-lettered. The protocol existed only as the behaviour of the Python implementation. It now has a version, shared constants and conformance cases that every library runs; the libraries passed 35, 107, 57 and 63 of their cases. My answer is the one I would give about any interface: the specification has to be an artefact the second implementation is tested against, or the first implementation silently *is* the specification.

Q33 (Dr. Yegge)

> *You compare against other engines and publish reproductions of other people's results. How do I know those comparisons are fair?*

By looking for my own errors in them, which I have found and published.

- A GIAB comparison reported 242,958 variants "retired". 41% of them were on a chromosome the newer release does not cover; corrected, the carryover is 95.9%, not 93.2%. That page was wrong twice, both times with a correct number that meant something other than its label.
- My PyPSA-Earth port changed an upstream default and produced 36 buses with no country. I now claim it as plumbing, not as data parity.

Two habits came out of these. A comparison reports the level at which two outputs agree, and separates what is out of scope from what differs. And every port records what it did *not* verify. I would trust a comparison here to the extent that it states its scope. That is the standard I would ask the committee to hold it to.

End of defense transcript.